ISTANBUL KÜLTÜR UNIVERSITY

Analyzing Indoor Air Quality

Submitted By

Furkan Temizel 2100003964
Kaan Yegül 2000005031
Melis Yanık 2100005652

Department of Computer Engineering
İstanbul Kültür University

2024-2025 Fall

ABSTRACT

This project analyzes indoor air quality datasets in a traveler transit area at Brindisi Airport Italy. Using data collected over one year. It leverages IoT-based air monitoring systems to track gas concentrations, CO2 levels, temperature, humidity and particulate matter. This report is for the preprocessing and statistical analyses.

TABLE OF CONTENTS

LIST OF FIGURES

1. INTRODUCTION

1.1. Problem Statement

Ensuring healthy indoor air quality is critical since the increasing air travel and time spent in transit areas. The project focuses on monitoring and analyzing air quality in Brindisi Airport.

1.2. Project Purpose

The purpose of this project is to analyze the datasets and observe air quality trends in transit areas for future real-time monitoring systems using low cost IoT sensors.

1.3. Project Scope

This report covers preprocessing and statistical analysis of the air quality datasets.

1.4. Objectives and Success Criteria of the Project

- Preprocess the 3 datasets (3 CSV files ranging from 7M records to 11M records) to clean and normalize data.
- Identify outliers and handle them.
- Normalize the data for future use.
- Do some statistical analysis on the preprocessed and normalized data.

2. METHODOLOGY

2.5 OVERVİEW OF THE DATASET/MODEL

The dataset contains 16 columns with over 7M records. Variables include gas concentration, CO2 levels, temperature, humidity, particulate matter, and timestamps.

2.6 TOOLS AND TECHNOLOGY

- Python Libraries: Pandas, NumPy, Scikit-learn, Seaborn, Matplotlib
- Jupyter Notebook

2.7 PROPOSED APPROACH

- Preprocessing: Handle missing values, outliers, and type conversions.
- Normalization: Apply Min-Max scaling to prepare data for analysis.
- Statistical Analysis: Identify trends in air quality metrics.

3. EXPERIMENTAL RESULTS

3.1 PREPROCESSING

1. Loading and Inspecting the Dataset(s)

We have 3 different csv files for 3 different sensors' data held which is the measurements of air quality records of each sensor. We are doing the steps on ony the complete_measure_node_airport_gold.csv to apply them to the others later on.

We observe that it separates values using ' ; ' instead of ' , '

```
In [17]: import pandas as pd
         import numpy as np
         import seaborn as sns
         from sklearn.preprocessing import StandardScaler

         df = pd.read_csv("D:\complete_measure_node_airport_gold.csv")

         df
```

Out[17]:

m25_ug;sps30_pm4_count;sps30_pm4_ug;sps30_pm10_count;sps30_pm10_ug;sps30_pm_typ;ts_insertion
1;node-air-gold;1950;0.243757;606.316;29.3313;...
2;node-air-gold;1997;0.249633;605.303;29.3019;...
3;node-air-gold;1997;0.249633;605.58;29.3153;5...
4;node-air-gold;1977;0.247133;605.191;29.3153;...
5;node-air-gold;1957;0.244632;603.975;29.3019;...
...
7490026;node-air-gold;1663;0.207881;713.675;28...
7490027;node-air-gold;1664;0.208006;713.536;28...
7490028;node-air-gold;1645;0.205631;713.131;28...
7490029;node-air-gold;1664;0.207881;711.359;28...
7490030;node-air-gold;1656;0.207006;710.733;28...

Figure 3.1.1: CSV file's values are separated by ';'

We define a delimiter while calling.

```
In [18]: df = pd.read_csv("D:\complete_measure_node_airport_gold.csv", delimiter=';')
         df
```

Out[18]:

	id_measure	node_id	ads1115_value	ads1115_voltage	scd30_co2	scd30_temp	scd30_hur
0	1	node-air-gold	1950	0.243757	606.316	29.3313	51.725
1	2	node-air-gold	1997	0.249633	605.303	29.3019	51.669
2	3	node-air-gold	1997	0.249633	605.580	29.3153	51.632
3	4	node-air-gold	1977	0.247133	605.191	29.3153	51.724
4	5	node-air-gold	1957	0.244632	603.975	29.3019	51.655
...	...	...	...	...	...	...	.
7105446	7490026	node-air-gold	1663	0.207881	713.675	28.1483	43.406
7105447	7490027	node-air-gold	1664	0.208006	713.536	28.1483	43.379
7105448	7490028	node-air-gold	1645	0.205631	713.131	28.1617	43.324
7105449	7490029	node-air-gold	1664	0.207881	711.359	28.1617	43.365
7105450	7490030	node-air-gold	1656	0.207006	710.733	28.1483	43.392

7105451 rows × 18 columns

Figure 3.1.2: Dataset called by the correct delimiter ';'

Now we can load and inspect the datasets for preprocessing without any problem.

2. Checking and Handling NULL Values

We check for the null values using the code line "print(df.isnull().sum())" and we observe that there are some columns that have null values.

```
In [19]: print(df.isnull().sum()) #checking for NULL values in the dataset
```

```
id_measure           0
node_id              0
ads1115_value        0
ads1115_voltage      0
scd30_co2            0
scd30_temp           0
scd30_hum            0
sps30_pm05_count    12
sps30_pm1_count     15
sps30_pm1_ug         0
sps30_pm25_count    17
sps30_pm25_ug        8
sps30_pm4_count     23
sps30_pm4_ug         9
sps30_pm10_count    23
sps30_pm10_ug       11
sps30_pm_typ        25
ts_insertion         0
dtype: int64
```

Figure 3.1.3: Finding columns with NULL values

We use this code to replace all the null values in the datasets to 0.

```
In [20]: #Turning null values into 0's
columns_to_fill = [
    'sps30_pm05_count', 'sps30_pm1_count', 'sps30_pm25_count', 'sps30_pm4_count'
    'sps30_pm10_count', 'sps30_pm_typ', 'sps30_pm25_ug', 'sps30_pm4_ug', 'sps30_
]

df[columns_to_fill] = df[columns_to_fill].fillna(0)

print(df.isnull().sum())
```

```
id_measure          0
node_id             0
ads1115_value       0
ads1115_voltage     0
scd30_co2           0
scd30_temp          0
scd30_hum           0
sps30_pm05_count    0
sps30_pm1_count     0
sps30_pm1_ug        0
sps30_pm25_count    0
sps30_pm25_ug       0
sps30_pm4_count     0
sps30_pm4_ug        0
sps30_pm10_count    0
sps30_pm10_ug       0
sps30_pm_typ        0
ts_insertion        0
dtype: int64
```

Figure 3.1.4: NULL values cleaned from the columns

3. Dropping Irrelevant Columns

We drop these columns: id_measure, node_id and observe the dataset afterwards.

```
In [21]: cols_to_drop=['id_measure','node_id']

df = df.drop(columns=df.filter(items=cols_to_drop).columns)

df
```

Out[21]:

	ads1115_value	ads1115_voltage	scd30_co2	scd30_temp	scd30_hum	sps30_pm05_count
0	1950	0.243757	606.316	29.3313	51.7258	26.4676
1	1997	0.249633	605.303	29.3019	51.6693	26.3993
2	1997	0.249633	605.580	29.3153	51.6327	26.4615
3	1977	0.247133	605.191	29.3153	51.7242	26.4615
4	1957	0.244632	603.975	29.3019	51.6556	26.4615
...	...	...	...	...	...	...
7105446	1663	0.207881	713.675	28.1483	43.4067	38.6570
7105447	1664	0.208006	713.536	28.1483	43.3792	39.3510
7105448	1645	0.205631	713.131	28.1617	43.3243	39.4641
7105449	1664	0.207881	711.359	28.1617	43.3655	39.8152
7105450	1656	0.207006	710.733	28.1483	43.3929	40.1366

7105451 rows × 16 columns

Figure 3.1.5: Dataset after dropping columns

4. Converting Data Types

We observe the data types of the columns with df.info()

```
In [23]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7105451 entries, 0 to 7105450
Data columns (total 16 columns):
 #   Column            Dtype
---  ------            -----
 0   ads1115_value     int64
 1   ads1115_voltage   float64
 2   scd30_co2         float64
 3   scd30_temp        float64
 4   scd30_hum         float64
 5   sps30_pm05_count  float64
 6   sps30_pm1_count   float64
 7   sps30_pm1_ug      float64
 8   sps30_pm25_count  float64
 9   sps30_pm25_ug     float64
 10  sps30_pm4_count   float64
 11  sps30_pm4_ug      float64
 12  sps30_pm10_count  float64
 13  sps30_pm10_ug     float64
 14  sps30_pm_typ      float64
 15  ts_insertion      object
dtypes: float64(14), int64(1), object(1)
memory usage: 867.4+ MB
```

Figure 3.1.6: Observing data types of the columns

We convert the one integer column to float64 and convert the “ts_insertion” object to datetime

```
In [26]: df['ts_insertion'] = pd.to_datetime(df['ts_insertion'])

int_cols = df.select_dtypes(include=['int64']).columns
df[int_cols] = df[int_cols].astype('float64')

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7105451 entries, 0 to 7105450
Data columns (total 16 columns):
 #   Column            Dtype
---  ------            -----
 0   ads1115_value     float64
 1   ads1115_voltage   float64
 2   scd30_co2         float64
 3   scd30_temp        float64
 4   scd30_hum         float64
 5   sps30_pm05_count  float64
 6   sps30_pm1_count   float64
 7   sps30_pm1_ug      float64
 8   sps30_pm25_count  float64
 9   sps30_pm25_ug     float64
 10  sps30_pm4_count   float64
 11  sps30_pm4_ug      float64
 12  sps30_pm10_count  float64
 13  sps30_pm10_ug     float64
 14  sps30_pm_typ      float64
 15  ts_insertion      datetime64[ns]
dtypes: datetime64[ns](1), float64(15)
memory usage: 867.4 MB
```

Figure 3.1.7: Data Types after data conversion

5. Finding and Handling Outliers

First, we find how many outliers each column has by using IQR calculation.

```
In [225]: numeric_cols = df.select_dtypes(include=['float64', 'int64'])

          Q1 = numeric_cols.quantile(0.25)
          Q3 = numeric_cols.quantile(0.75)
          IQR = Q3 - Q1

          lower_bound = Q1 - 5 * IQR
          upper_bound = Q3 + 5 * IQR

          outliers = (numeric_cols < lower_bound) | (numeric_cols > upper_bound)

          outliers_count = outliers.sum()
          print(outliers_count)

          ads1115_value         3041
          ads1115_voltage       3036
          scd30_co2             2952
          scd30_temp           24944
          scd30_hum                0
          sps30_pm05_count     46656
          sps30_pm1_count      46573
          sps30_pm1_ug         46577
          sps30_pm25_count     46596
          sps30_pm25_ug        55886
          sps30_pm4_count      46606
          sps30_pm4_ug         61050
          sps30_pm10_count     46615
          sps30_pm10_ug        61555
          sps30_pm_typ        105851
          dtype: int64
```

Figure 3.1.8: Finding how many outliers each column has using IQR calculation

```
plt.figure(figsize=(12, 8))
numeric_cols = df2.select_dtypes(include=['float64', 'int64']).columns

for i, col in enumerate(numeric_cols, 1):
    plt.subplot(4, 4, i)
    sns.boxplot(data=df, x=col)
    plt.title(f"Boxplot of {col}")

plt.tight_layout()
plt.show()
```

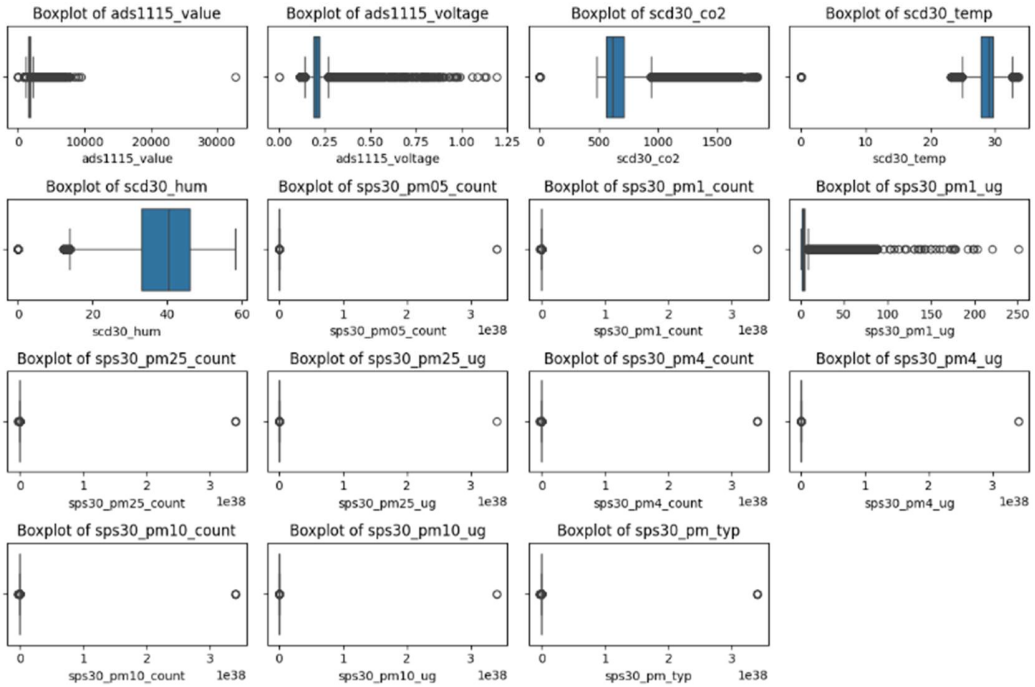

Figure 3.1.9: Boxplot visualization before outlier handling

We handle the outliers by replacing them with the lower or the upper bound determined by the calculation. After the calculation we visualize them in box plots.

```
def handle_outliers_iqr(data, columns):
    data_cleaned = data.copy()
    for col in columns:
        Q1 = data_cleaned[col].quantile(0.25)
        Q3 = data_cleaned[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        data_cleaned[col] = data_cleaned[col].clip(lower=lower_bound, upper=upper_bound)
    return data_cleaned
```

```
#boxplots after iqr
cleaned_data = handle_outliers_iqr(df, numeric_cols)
plt.figure(figsize=(12, 8))
for i, col in enumerate(numeric_cols, 1):
    plt.subplot(4, 4, i)
    sns.boxplot(data=cleaned_data, x=col)
    plt.title(f"Boxplot of {col}")

plt.tight_layout()
plt.show()
```

Figure 3.1.10: The code for calculation and visualization

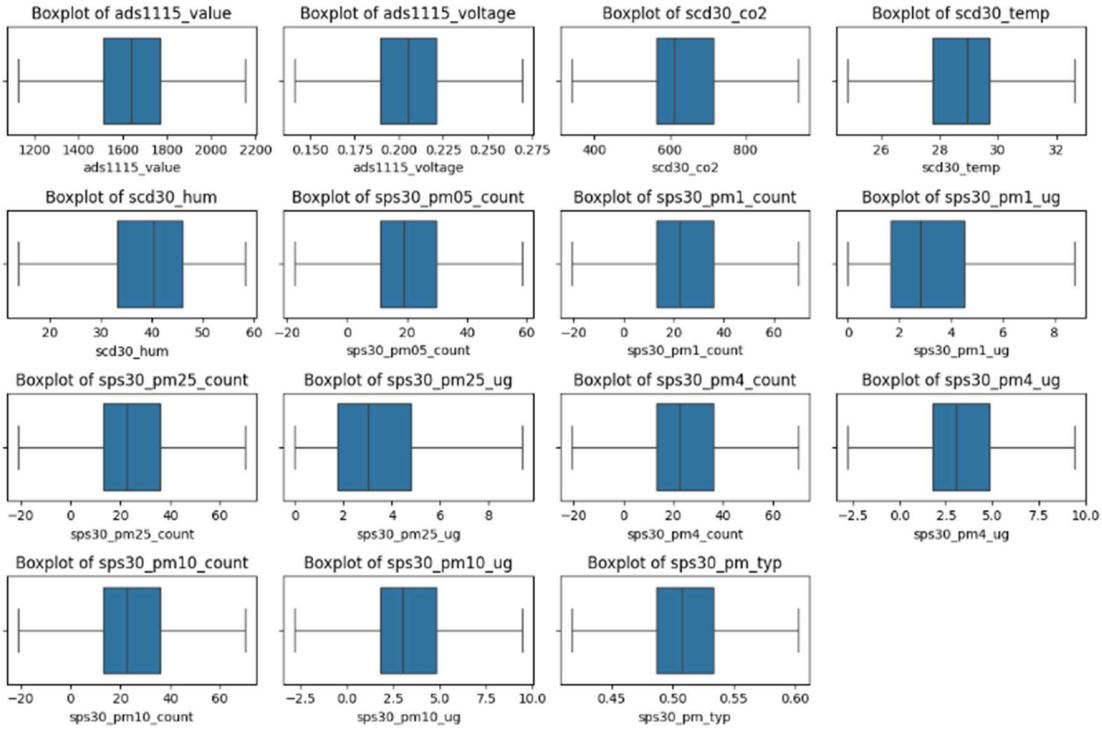

Figure 3.1.11: Boxplots after outlier handling

From the start, we used only one dataset for these processes out of the 3. Now we'll create a dataset that is the combination of all three datasets. We turn null values into 0 and drop columns.

```
import pandas as pd

#taking chunks of data instead of all at once to evade memory problems
chunk_size = 50000
chunks = []

for filename in ["C:\\Users\\Furkan\\Desktop\\complete_measure_node_airport_brown.csv",
                 "C:\\Users\\Furkan\\Desktop\\complete_measure_node_airport_silver.csv",
                 "C:\\Users\\Furkan\\Desktop\\complete_measure_node_airport_gold.csv"]:
    chunk_iter = pd.read_csv(filename, delimiter=';', chunksize=chunk_size)
    for chunk in chunk_iter:
        chunks.append(chunk)

df = pd.concat(chunks, ignore_index=True)
```

Figure 3.1.12: Loading all datasets and combining them into one

[2]:
```
df
```

[2]:

	id_measure	node_id	ads1115_value	ads1115_voltage	scd30_co2	scd30_temp	scd30_hum	sps30_pm05_count	sps30_pm1_count	sps30_pm1_ug	sps30_pm2
0	1	node-air-brown	2015	0.251883	659.087	29.3526	51.7715	43.0549	50.3810	6.32602	
1	2	node-air-brown	1950	0.243757	658.184	29.3660	51.7990	43.5107	50.9143	6.39299	
2	3	node-air-brown	1995	0.249133	655.991	29.3927	51.8387	44.1547	51.6679	6.48762	
3	4	node-air-brown	2010	0.251258	655.740	29.3526	51.8906	44.2329	51.7594	6.49910	
4	5	node-air-brown	1993	0.249133	655.774	29.3660	51.8448	44.4443	52.0068	6.53017	
...	...	...	...	...	...	...	...	...	...	...	
28177931	7490026	node-air-gold	1663	0.207881	713.675	28.1483	43.4067	38.6570	46.0142	5.80072	
28177932	7490027	node-air-gold	1664	0.208006	713.536	28.1483	43.3792	39.3510	46.8404	5.90487	
28177933	7490028	node-air-gold	1645	0.205631	713.131	28.1617	43.3243	39.4641	46.9749	5.92183	
28177934	7490029	node-air-gold	1664	0.207881	711.359	28.1617	43.3655	39.8152	47.3929	5.97452	
28177935	7490030	node-air-gold	1656	0.207006	710.733	28.1483	43.3929	40.1366	47.7755	6.02275	

28177936 rows × 18 columns

Figure 3.1.13: All datasets combined

```
print(df.isnull().sum())
```

```
id_measure            0
node_id               0
ads1115_value         0
ads1115_voltage       0
scd30_co2             0
scd30_temp            0
scd30_hum             0
sps30_pm05_count     75
sps30_pm1_count      85
sps30_pm1_ug          1
sps30_pm25_count     95
sps30_pm25_ug        52
sps30_pm4_count     105
sps30_pm4_ug         54
sps30_pm10_count    109
sps30_pm10_ug        64
sps30_pm_typ        120
ts_insertion          0
dtype: int64
```

```
#turning null values into 0s
columns_to_fill = [
    'sps30_pm05_count', 'sps30_pm1_count', 'sps30_pm25_count', 'sps30_pm4_count',
    'sps30_pm10_count', 'sps30_pm_typ', 'sps30_pm25_ug', 'sps30_pm4_ug', 'sps30_pm10_ug'
]

df[columns_to_fill] = df[columns_to_fill].fillna(0)

print(df.isnull().sum())
```

```
node_id             0
ads1115_value       0
ads1115_voltage     0
scd30_co2           0
scd30_temp          0
scd30_hum           0
sps30_pm05_count    0
sps30_pm1_count     0
sps30_pm1_ug        1
sps30_pm25_count    0
sps30_pm25_ug       0
sps30_pm4_count     0
sps30_pm4_ug        0
sps30_pm10_count    0
sps30_pm10_ug       0
sps30_pm_typ        0
ts_insertion        0
dtype: int64
```

Figure 3.1.14: Clearing null values

We use one-hot encoding to create separate binary columns for each dataset where each column indicates which category a row belongs to. This helps distinguish between the 'brown', 'gold' and 'silver' datasets for easier analysis.

```
df['brown'] = (df['node_id'] == 'node-air-brown').astype(int)
df['gold'] = (df['node_id'] == 'node-air-gold').astype(int)
df['silver'] = (df['node_id'] == 'node-air-silver').astype(int)

df = df.drop(columns=['id_measure'])
df = df.drop(columns=['node_id'])

df['ts_insertion'] = pd.to_datetime(df['ts_insertion'])

int_cols = df.select_dtypes(include=['int64']).columns
df[int_cols] = df[int_cols].astype('float64')

df.info()
```

Figure 3.1.15: One-hot encoding, dropping columns and changing datatypes

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28177936 entries, 0 to 28177935
Data columns (total 19 columns):
 #   Column            Dtype
---  ------            -----
 0   ads1115_value     float64
 1   ads1115_voltage   float64
 2   scd30_co2         float64
 3   scd30_temp        float64
 4   scd30_hum         float64
 5   sps30_pm05_count  float64
 6   sps30_pm1_count   float64
 7   sps30_pm1_ug      float64
 8   sps30_pm25_count  float64
 9   sps30_pm25_ug     float64
 10  sps30_pm4_count   float64
 11  sps30_pm4_ug      float64
 12  sps30_pm10_count  float64
 13  sps30_pm10_ug     float64
 14  sps30_pm_typ      float64
 15  ts_insertion      datetime64[ns]
 16  brown             float64
 17  gold              float64
 18  silver            float64
dtypes: datetime64[ns](1), float64(18)
memory usage: 4.0 GB
```

Figure 3.1.16: Datatypes and columns of the combined dataset

We again handle the outliers by replacing them with the lower or the upper bound determined by the calculation. After the calculation we visualize them in box plots.

```python
def handle_outliers_iqr(data, columns):
    data_cleaned = data.copy()
    for col in columns:
        Q1 = data_cleaned[col].quantile(0.25)
        Q3 = data_cleaned[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        data_cleaned[col] = data_cleaned[col].clip(lower=lower_bound, upper=upper_bound)
    return data_cleaned
```

```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns
cleaned_data = handle_outliers_iqr(df, numeric_cols)

numeric_cols = [col for col in numeric_cols if col not in ['brown', 'silver', 'gold']]

plt.figure(figsize=(12, 8))

for i, col in enumerate(numeric_cols, 1):
    plt.subplot(4, 4, i)
    sns.boxplot(data=cleaned_data, x=col)
    plt.title(f"Boxplot of {col}", fontsize=10)
    plt.xticks(fontsize=8)

plt.subplots_adjust(hspace=0.6, wspace=0.4) #spacing each plot so they don't collide with one anohter

plt.tight_layout()
plt.show()
```

Figure 3.1.17: Outlier handling for the combined dataset

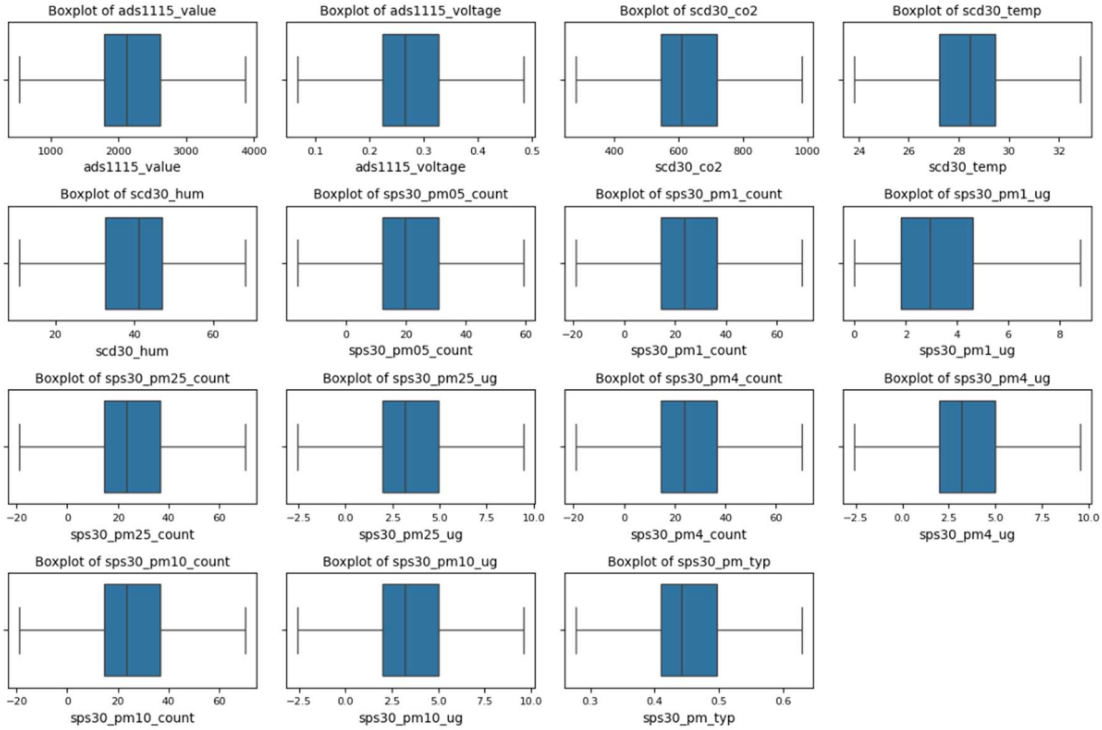

Figure 3.1.18: Boxplots after outlier handling for the combined dataset

3.2 STATISTICAL ANALYSES

1. Correlation

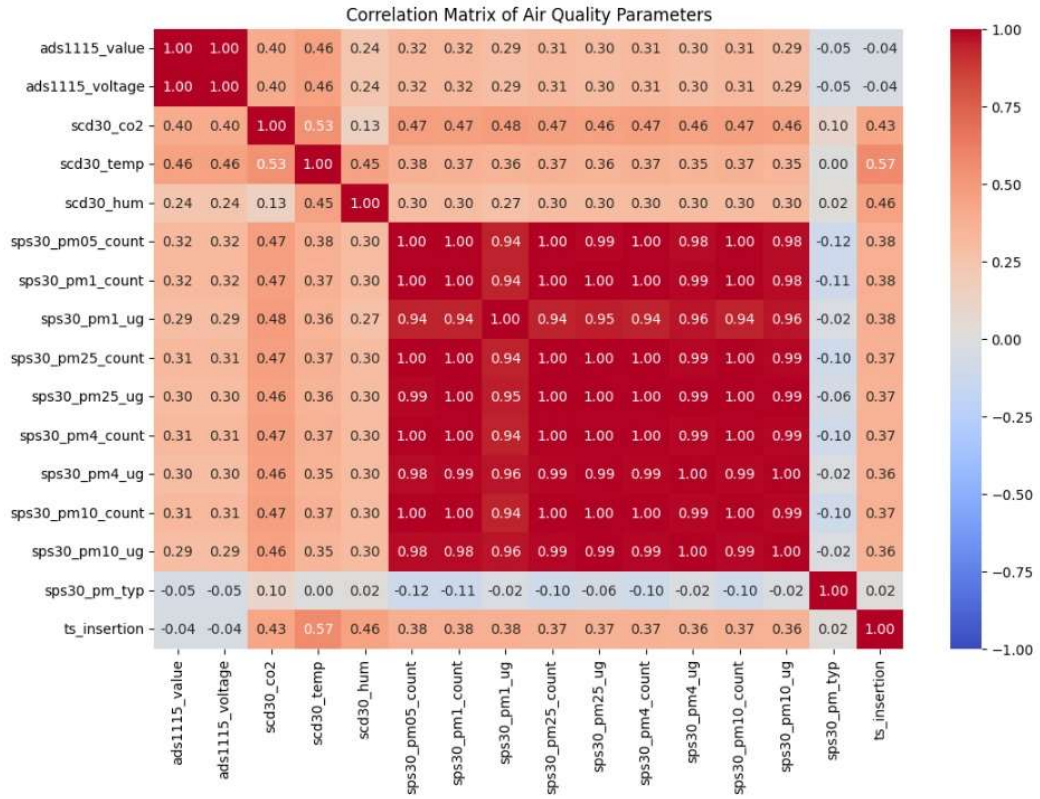

Figure 3.2.1: Correlation Heatmap of the columns

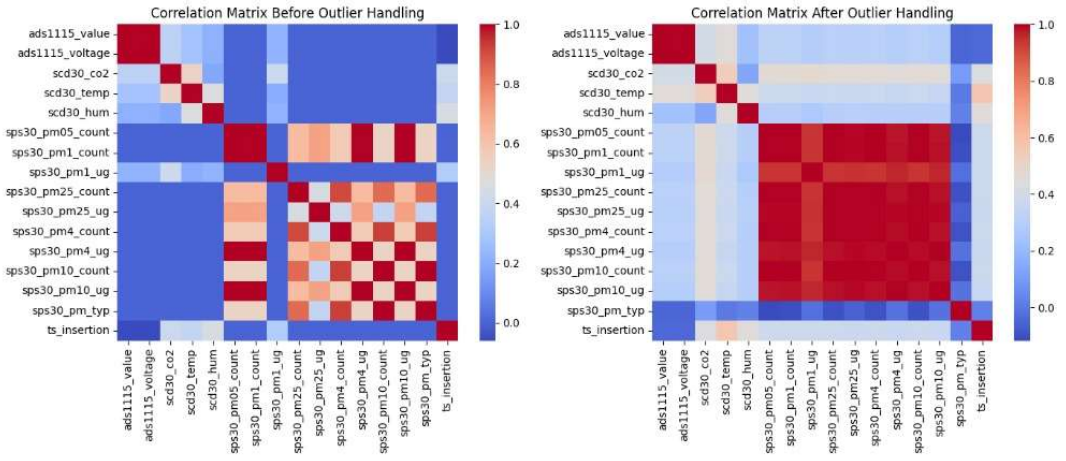

Figure 3.2.2: Correlation Heatmap comparison of the columns before and after outlier handling

```
corr2 = cleaned_data.corr()
plt.figure(figsize=(12, 8))
sns.heatmap(corr2, annot=True, cmap='coolwarm', fmt='.2f', vmin=-1, vmax=1)
plt.title('Correlation Matrix of Air Quality Parameters')
plt.show()
```

Figure 3.2.3: Correlation heatmap code

```
columns_to_exclude = ['brown', 'silver', 'gold']
filtered_original = df.drop(columns=columns_to_exclude, errors='ignore')
filtered_cleaned = cleaned_data.drop(columns=columns_to_exclude, errors='ignore')

correlation_original = filtered_original.corr()
correlation_cleaned = filtered_cleaned.corr()

plt.figure(figsize=(14, 6))

#Original data correlation matrix
plt.subplot(1, 2, 1)
sns.heatmap(correlation_original, annot=False, cmap='coolwarm')
plt.title('Correlation Matrix Before Outlier Handling')

#Cleaned data correlation matrix
plt.subplot(1, 2, 2)
sns.heatmap(correlation_cleaned, annot=False, cmap='coolwarm')
plt.title('Correlation Matrix After Outlier Handling')

plt.tight_layout()
plt.show()
```

Figure 3.2.4: Correlation heatmap comparison code

In our statistical analysis we used correlation heatmaps to visualize the relationships between columns in the dataset. We compared heatmaps before and after handling outliers to see how outlier removal affected these relationships.

2. Temperature – CO2 Scatterplot

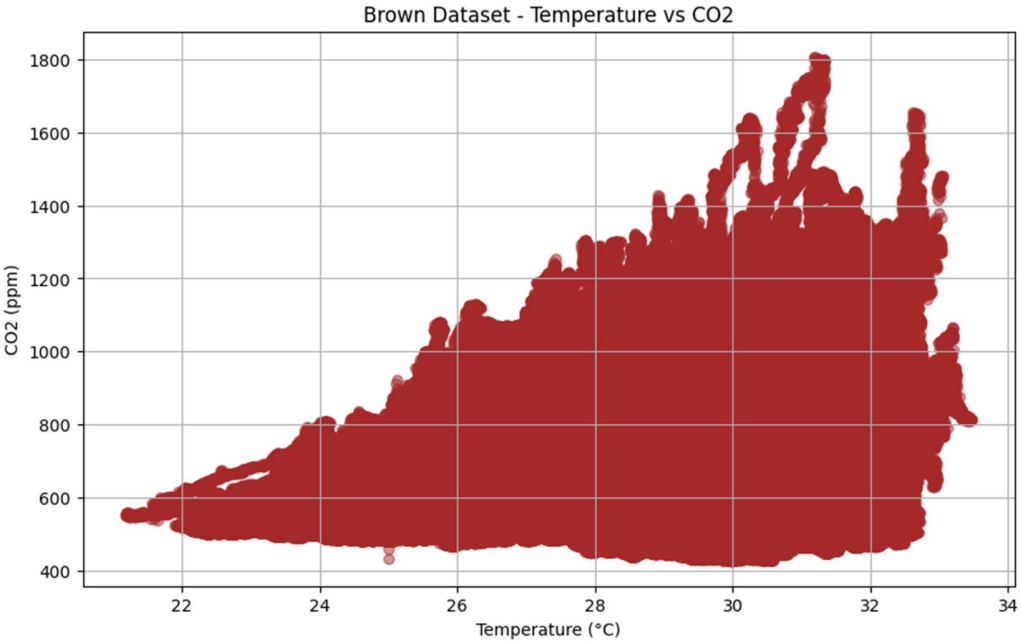

Figure 3.2.5: Temp/CO2 Scatterplot for Brown dataset visualization

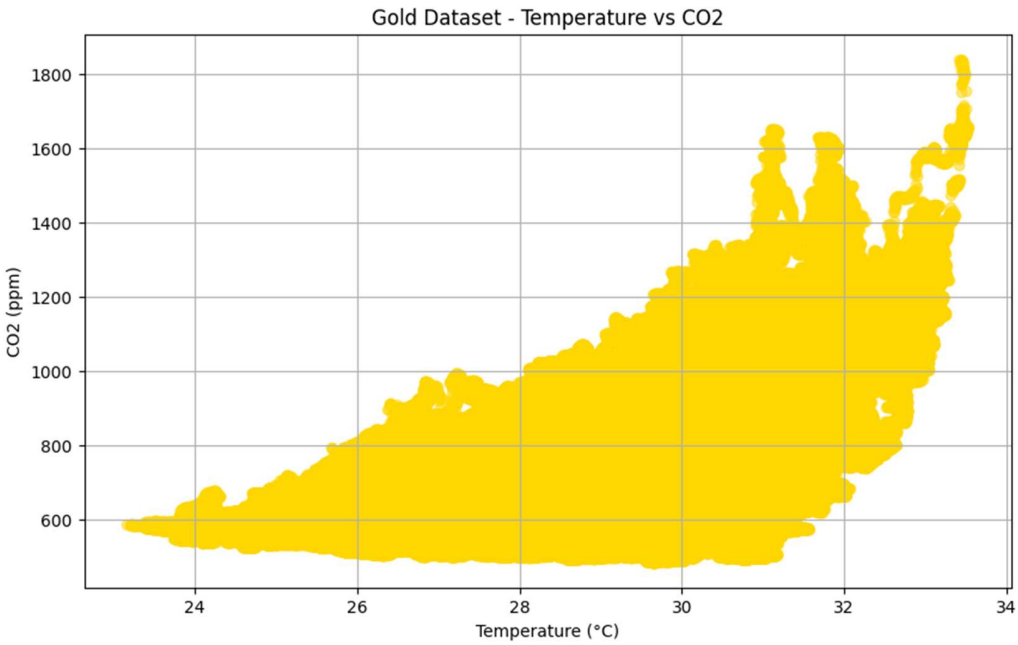

Figure 3.2.6: Temp/CO2 Scatterplot for Gold dataset visualization

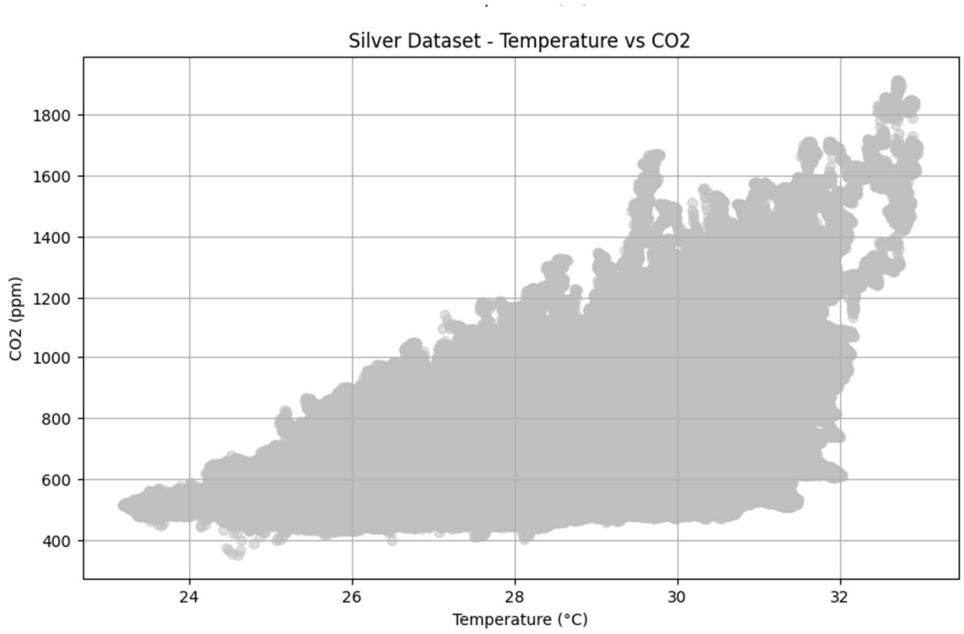

Figure 3.2.7: Temp/CO2 Scatterplot for Silver dataset visualization

```python
import pandas as pd
import matplotlib.pyplot as plt

#Filtering
df_brown = df[df['brown'] == 1]
df_gold = df[df['gold'] == 1]
df_silver = df[df['silver'] == 1]

#Scatter plot for Brown data
plt.figure(figsize=(10, 6))
plt.scatter(df_brown['scd30_temp'], df_brown['scd30_co2'], color='brown', alpha=0.5)
plt.title('Brown Dataset - Temperature vs CO2')
plt.xlabel('Temperature (°C)')
plt.ylabel('CO2 (ppm)')
plt.grid(True)
plt.show()

#Scatter plot for Gold data
plt.figure(figsize=(10, 6))
plt.scatter(df_gold['scd30_temp'], df_gold['scd30_co2'], color='gold', alpha=0.5)
plt.title('Gold Dataset - Temperature vs CO2')
plt.xlabel('Temperature (°C)')
plt.ylabel('CO2 (ppm)')
plt.grid(True)
plt.show()

#Scatter plot for Silver data
plt.figure(figsize=(10, 6))
plt.scatter(df_silver['scd30_temp'], df_silver['scd30_co2'], color='silver', alpha=0.5)
plt.title('Silver Dataset - Temperature vs CO2')
plt.xlabel('Temperature (°C)')
plt.ylabel('CO2 (ppm)')
plt.grid(True)
plt.show()
```

Figure 3.2.8: Code for Scatterplot visualization

The charts display the relationship between temperature and CO2 levels for three distinct datasets: Brown, Gold and Silver. Each scatter plot highlights how CO2 concentrations vary with temperature within each group. It can be analyzed that the CO2 amount increases with temperature increase.

3. Temperature – Humidity Scatterplot

```
import matplotlib.pyplot as plt
import seaborn as sns

filtered_data = df[(df['scd30_temp'] >= 20) & (df['scd30_temp'] <= 35)]

plt.figure(figsize=(12, 8))

sns.scatterplot(
    data=filtered_data[filtered_data['brown'] == 1],
    x='scd30_temp',
    y='scd30_hum',
    label='Brown',
    color='blue'
)

sns.scatterplot(
    data=filtered_data[filtered_data['gold'] == 1],
    x='scd30_temp',
    y='scd30_hum',
    label='Gold',
    color='orange'
)

sns.scatterplot(
    data=filtered_data[filtered_data['silver'] == 1],
    x='scd30_temp',
    y='scd30_hum',
    label='Silver',
    color='green'
)

plt.title('Temperature vs Humidity for Brown, Gold, and Silver')
plt.xlabel('Temperature (°C)')
plt.ylabel('Humidity (%)')

plt.legend()
plt.show()
```

Figure 3.2.9: Code for Scatterplot visualization

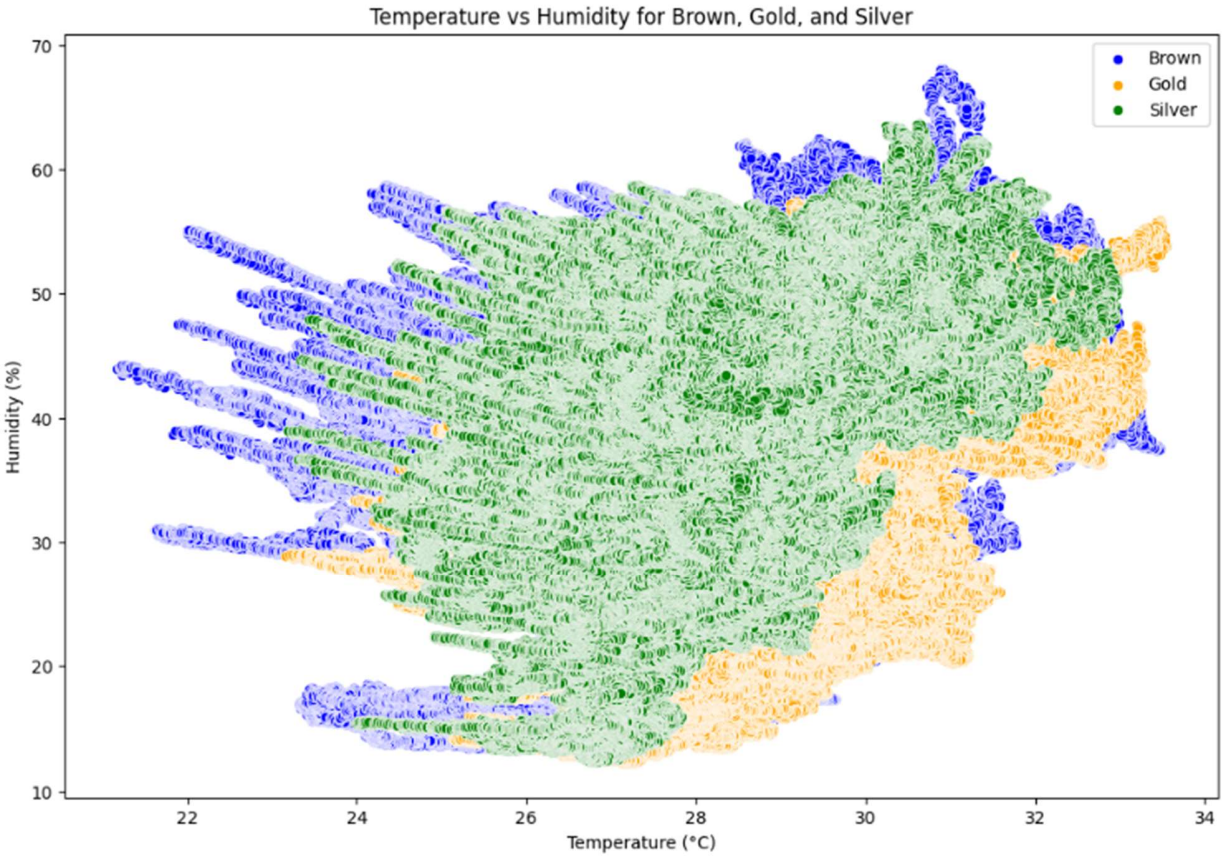

Figure 3.2.10: Temperature – Humidity Scatterplot of all data categorized by different datasets

This scatter plot visualizes the relationship between temperature and humidity for three different datasets (Brown, Gold and Silver). The scd30_temp column represents temperature in degrees Celsius while scd30_hum represents humidity percentage. Data points are color-coded by dataset: blue for Brown, orange for Gold and green for Silver.

4. CO2 – Time Plot

```
cleaned_data['ts_insertion'] = pd.to_datetime(cleaned_data['ts_insertion'])
df_sorted = df.sort_values('ts_insertion')
df_sorted['quarter'] = df_sorted['ts_insertion'].dt.to_period('Q')
unique_quarters = df_sorted['quarter'].unique()

for quarter in unique_quarters:
    quarter_data = df_sorted[df_sorted['quarter'] == quarter]
    plt.figure(figsize=(14, 6))
    plt.plot(quarter_data['ts_insertion'], quarter_data['scd30_co2'], label=f'CO2 Levels - {quarter}', color='blue')
    plt.xlabel('Timestamp')
    plt.ylabel('CO2 Level (ppm)')
    plt.title(f'CO2 Levels Over Time - {quarter}')
    plt.legend()
    plt.show()
```

Figure 3.2.11: Code for CO2 – Time Plot visualization

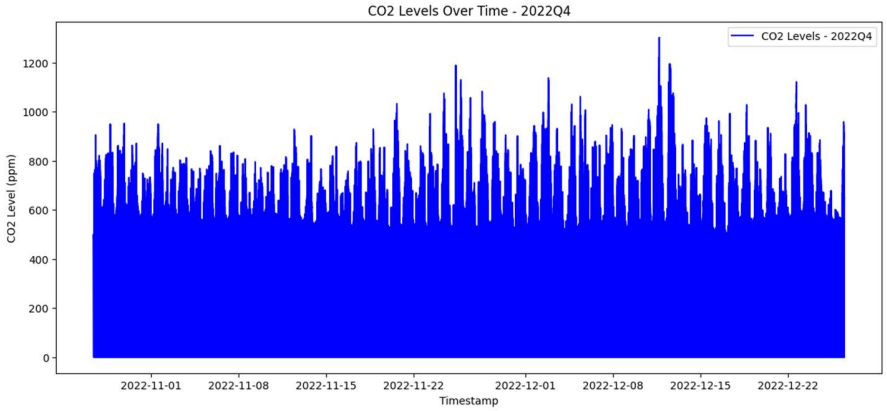

Figure 3.2.12: CO2 – Time Plot 2022Q4

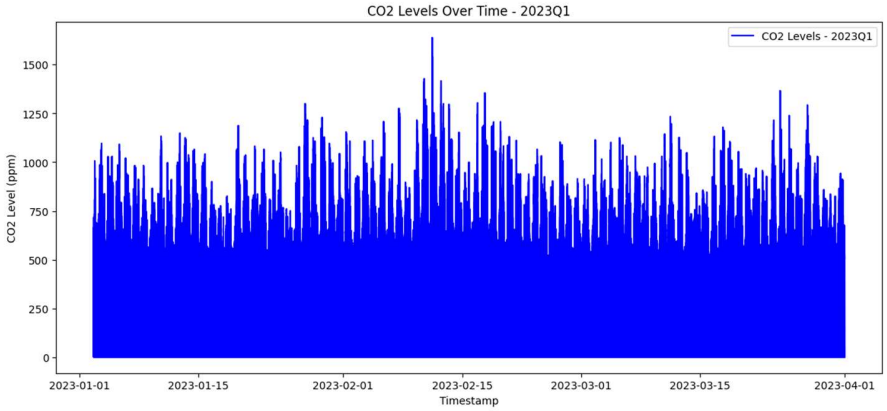

Figure 3.2.13: CO2 – Time Plot 2023Q1

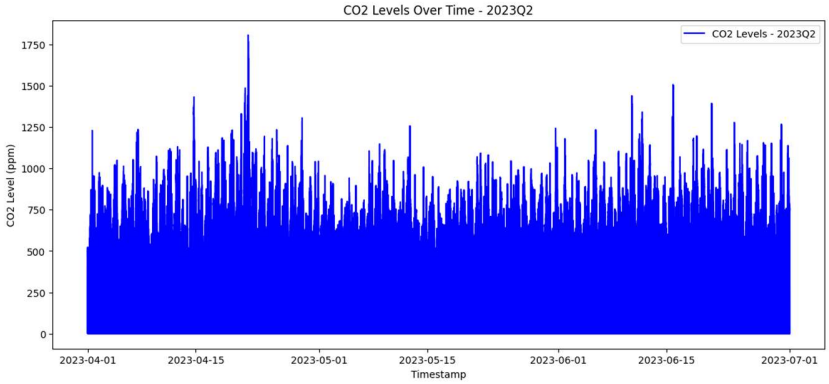

Figure 3.2.14: CO2 – Time Plot 2023Q2

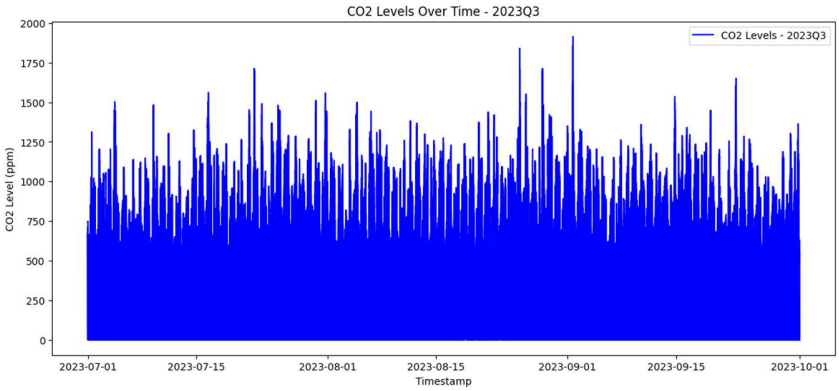

Figure 3.2.15: CO2 – Time Plot 2023Q3

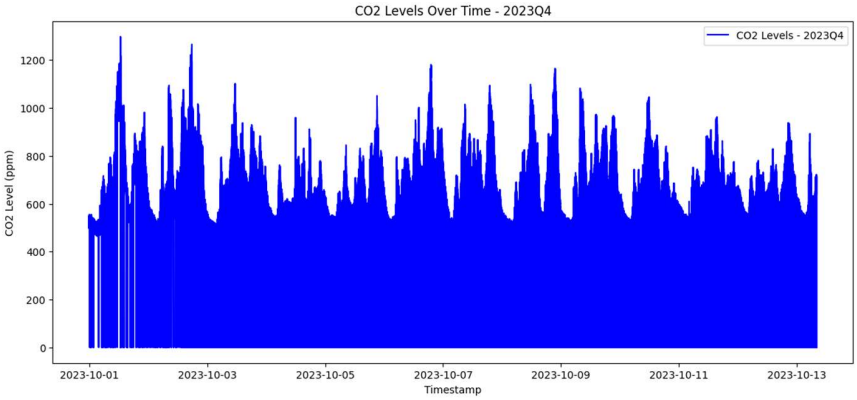

Figure 3.2.16: CO2 – Time Plot 2023Q4

The charts show CO2 levels over time for each yearly quarter. Each plot represents a different quarter and displays how CO2 levels change over time.

3.3 STATISTICAL TESTS

1. ANOVA

```
from scipy.stats import f_oneway

brown_group = df[df['brown'] == 1]['scd30_co2']
gold_group = df[df['gold'] == 1]['scd30_co2']
silver_group = df[df['silver'] == 1]['scd30_co2']

#ANOVA
f_stat, p_value = f_oneway(brown_group, gold_group, silver_group)

print("F-Statistic:", f_stat)
print("P-Value:", p_value)

if p_value < 0.05:
    print("There is a statistically significant difference between the groups.")
else:
    print("There is no statistically significant difference between the groups.")
```

```
F-Statistic: 80501.23348505184
P-Value: 0.0
There is a statistically significant difference between the groups.
```

Figure 3.3.1: ANOVA Code and Result

Null Hypothesis (H_0): There is no significant difference between the CO2 means of the three groups (Brown, Gold, and Silver).

Alternative Hypothesis (H_1): There is a significant difference between the CO2 means of at least one group (Brown, Gold, and Silver).

ANOVA Results Interpretation:

The F-statistic is 80501.23, and the p-value is 0.0.

Since the p-value is less than the significance level of 0.05 we reject the null hypothesis (H_0) and accept the alternative hypothesis (H_1). This indicates that there is a statistically significant difference in the CO2 means between the three groups (Brown, Gold, and Silver).

2. Sampling

2.1. Cluster Sampling

```
import numpy as np
import pandas as pd

#Cluster sampling
def cluster_sampling(df, number_of_clusters):
    try:
        cluster_size = len(df) // number_of_clusters
        remaining = len(df) % number_of_clusters
        cluster_ids = np.repeat(range(1, number_of_clusters + 1), cluster_size)         #creating the cluster_id column
        cluster_ids = np.concatenate([cluster_ids, np.repeat(range(1, remaining + 1), 1)])

        np.random.shuffle(cluster_ids)
        df['cluster_id'] = cluster_ids

        indexes = df[df['cluster_id'] % 2 == 0].index

        cluster_sample = df.loc[indexes]
        return cluster_sample
    except Exception as e:
        print(f"An error occurred: {e}")
        return pd.DataFrame()

cluster_sample = cluster_sampling(df, 5)

if cluster_sample.empty:
    print("No valid cluster sample was created.")
else:
    print("Cluster Sample CO2 Mean:", cluster_sample['scd30_co2'].mean())
```

```
Cluster Sample CO2 Mean: 647.0638533336452
```

Figure 3.3.2: Cluster Sampling Code and Result

2.2 Simple Random Sampling

```
from random import sample

co2_list = list(df[df['gold'] == 1]['scd30_co2'])

#simple random sampling (500 samples)
sample_co2 = sample(co2_list, 500)

print("Simple Random Sample CO2 Mean:", sum(sample_co2) / len(sample_co2))
```

```
Simple Random Sample CO2 Mean: 663.1795
```

Figure 3.3.3: Simple Random Sampling Code and Result

2.3 Stratified Sampling

```
#Stratified sampling
brown_sample = df[df['brown'] == 1].groupby('sps30_pm_typ', group_keys=False).apply(lambda x: x.sample(frac=0.1))
gold_sample = df[df['gold'] == 1].groupby('sps30_pm_typ', group_keys=False).apply(lambda x: x.sample(frac=0.1))
silver_sample = df[df['silver'] == 1].groupby('sps30_pm_typ', group_keys=False).apply(lambda x: x.sample(frac=0.1))

print("Brown Sample CO2 Mean:", brown_sample['scd30_co2'].mean())
print("Gold Sample CO2 Mean:", gold_sample['scd30_co2'].mean())
print("Silver Sample CO2 Mean:", silver_sample['scd30_co2'].mean())
```

```
Brown Sample CO2 Mean: 660.9622185450919
Gold Sample CO2 Mean: 660.0839188085181
Silver Sample CO2 Mean: 637.2300334685523
```

Figure 3.3.4: Stratified Sampling Code and Result

To evaluate the CO2 mean levels across different sampling methods we performed the following sampling techniques: Simple Random Sampling, Cluster Sampling, and Stratified Random Sampling. The CO2 mean values for each sampling method are as follows:

Simple Random Sample CO2 Mean: 663.1795

Cluster Sample CO2 Mean: 647.06

Stratified Random Sampling:
Brown Sample CO2 Mean: 660.96
Gold Sample CO2 Mean: 660.08
Silver Sample CO2 Mean: 637.23

Simple Random Sample showed the highest CO2 mean of 663.18 which suggests that this method may be more representative of the overall population CO2 levels.

Cluster Sample yielded a lower CO2 mean of 647.06 which could reflect a sampling bias, as only certain clusters were chosen based on specific criteria (even numbered clusters).

Stratified Random Sampling provided CO2 means for each dataset (Brown, Gold and Silver):
Brown Sample: 660.96
Gold Sample: 660.08
Silver Sample: 637.23
These results show differences across different datasets with the Silver sample having the lowest CO2 mean which might suggest differences in characteristics within each dataset.

Based on these results the Simple Random Sample method appears to give a higher average CO2 level compared to the other sampling techniques. However the Stratified Random Sampling method provides more detailed insights into the specific groups while the Cluster Sampling method yielded the lowest CO2 mean.

4. DISCUSSION

The project focused on cleaning and analyzing 3 datasets by handling outliers and merging files and doing some statistical analysis on them. Outliers were reduced using IQR calculations. This improves data quality without losing meaningful information. Visualizations and statistical tests confirmed the success of the approach. This work highlights practical methods for managing big data with applications.

REFERENCES

APPENDIX

Columns and their meaning in the datasets:

1. ads1115_value - ADC raw value
2. ads1115_voltage - ADC voltage reading
3. scd30_co2 - CO_2 concentration
4. scd30_temp - Temperature (°C)
5. scd30_hum - Humidity (%)
6. sps30_pm05_count - PM0.5 particle count
7. sps30_pm1_count - PM1.0 particle count
8. sps30_pm1_ug - PM1.0 mass (µg/m³)
9. sps30_pm25_count - PM2.5 particle count
10. sps30_pm25_ug - PM2.5 mass (µg/m³)
11. sps30_pm4_count - PM4.0 particle count
12. sps30_pm4_ug - PM4.0 mass (µg/m³)
13. sps30_pm10_count - PM10 particle count
14. sps30_pm10_ug - PM10 mass (µg/m³)
15. sps30_pm_typ - Typical particle size
16. ts_insertion - Timestamp of data